

Prueba Técnica

Desarrollador Full Stack

Objetivo

Desarrollar una aplicación funcional para la gestión de tareas que permita crear, visualizar, editar y eliminar tareas. Esta prueba evalúa tu capacidad para construir una solución full-stack con buenas prácticas de desarrollo, manejo de errores, validaciones y arquitectura escalable.

Tecnologías Exigidas

- Angular
- Node.js + Express (o framework similar)

Backend

Construir una API REST que permita realizar operaciones CRUD sobre tareas.

Modelo de Datos

- id (identificador único)
- title (string, obligatorio, 1-100 caracteres)
- description (string, opcional, máx 500 caracteres)
- status (pending | in_progress | done)
- createdAt (timestamp)
- updatedAt (timestamp)

Endpoints Esperados

- GET /api/tasks - Listar todas las tareas
- GET /api/tasks/:id - Obtener tarea por ID
- POST /api/tasks - Crear nueva tarea
- PUT /api/tasks/:id - Actualizar tarea
- DELETE /api/tasks/:id - Eliminar tarea

Requisitos Obligatorios - Backend

- Uso correcto de métodos HTTP (GET, POST, PUT, DELETE)
- Códigos de estado HTTP apropiados:
 - 200 OK - Operación exitosa
 - 201 Created - Recurso creado
 - 400 Bad Request - Datos inválidos
 - 404 Not Found - Recurso no existe
 - 500 Internal Server Error - Error del servidor
- Validación de datos en el servidor:
 - Validar campos obligatorios (title, status)
 - Validar longitudes máximas
 - Validar formato del status (enum)
 - Retornar mensajes de error descriptivos
- Manejo de errores robusto con try-catch
- Uso de base de datos en memoria, localStorage o SQLite
- Estructura clara de carpetas: controllers, services, models, middleware

Frontend

Desarrollar una interfaz en Angular que consuma la API y permita gestionar tareas de forma intuitiva.

Funcionalidades Mínimas

- Listar tareas con visualización clara
- Crear tarea mediante formulario
- Editar tarea existente
- Eliminar tarea
- Cambiar estado de tarea (pending → in_progress → done)

Requisitos Técnicos - Frontend

- Servicios Angular para consumo de API (HttpClient)
- Separación de componentes (componentes reutilizables)
- Formularios Reactivos con validaciones
- Manejo de errores HTTP:
 - Mostrar mensajes de error al usuario
 - Diferenciar entre tipos de error (404, 500, timeout, etc)
- Uso de Observables para manejo asíncronico
- Tipado fuerte con interfaces TypeScript
- Separación de responsabilidades (smart/dumb components)

Criterios de Evaluación

- Estructura y organización del proyecto (carpetas, archivos)
- Correcto uso de métodos HTTP
- Correcto uso de códigos de estado HTTP
- Exposición y consumo de servicios web REST
- Construcción y comunicación entre componentes Angular
- Construcción y uso de servicios en Angular
- Validación de datos en backend y frontend
- Manejo de errores y casos edge
- Claridad y limpieza del código
- Documentación y README completo

Puntos Extra (Jerarquizados)

- Indicadores de loading durante peticiones HTTP
- Mensajes de confirmación antes de eliminar
- Mensajes de éxito/error amigables al usuario
- Variables de entorno (.env) para configuración (puerto, URL base API)
- Ejemplos de request/response en README o comentarios JSDoc
- Uso de modales para crear/editar tareas
- Tests unitarios (backend: controladores/servicios o frontend: componentes/servicios)
- Autenticación básica (mock token o simple validación en headers)
- UI/UX cuidada y responsive (CSS, Bootstrap, Tailwind, etc)
- Documentación de API con ejemplos (Swagger o comentarios detallados)
- Filtros o búsqueda en el listado de tareas
- Persistencia de datos (base de datos relacional, Firebase, etc)

Entregables

Subir el proyecto a un repositorio público de GitHub con:

- Código fuente del backend
- Código fuente del frontend
- Archivo .gitignore apropiado

README.md con secciones obligatorias:

- Descripción del proyecto y objetivo
- Tecnologías utilizadas (versiones)
- Instrucciones de instalación paso a paso (dependencias, variables de entorno)
- Instrucciones de ejecución para desarrollo (puertos, URLs)
- Breve explicación de la arquitectura:
 - Estructura de carpetas del backend
 - Estructura de carpetas del frontend
 - Cómo se comunican backend y frontend
- Decisiones técnicas principales (por qué elegiste X tecnología, por qué esa estructura)
- Funcionalidades implementadas (incluyendo puntos extra si aplica)
- Posibles mejoras futuras (qué harías diferente con más tiempo)

Notas Importantes

- No es necesario implementar autenticación a menos que sea punto extra
- La base de datos puede ser en memoria, localStorage o SQLite
- Se valorará la calidad sobre la cantidad. Mejor código limpio que muchas características
- Incluir comentarios en código complejo o decisiones no obvias
- El tiempo estimado es 4-6 horas de desarrollo

Checklist Final

Antes de enviar, verifica que:

- El repositorio es público y accesible
- El código está limpio y sin archivos innecesarios
- El README es completo y las instrucciones funcionan
- La aplicación corre sin errores en desarrollo
- Todos los endpoints funcionan correctamente
- El frontend se comunica correctamente con el backend
- El manejo de errores es robusto
- Hay validaciones en backend y frontend